

NLP 발전 흐름 한 번에 이해하기

워드 임베딩 -> RNN/LSTM -> Seq2Seq/Attention -> Transformer -> BERT/GPT



GOAL 이 자료의 목표

개념을 따로 외우는 대신, "이전 방법의 문제 -> 다음 방법의 해결책"이라는 한 줄의 발전 스토리로 연결한다. 수식은 최소화하고, 직관과 비교를 중심으로 이해한다.

대상: 딥러닝/NLP 입문자 | 기준 자료: 손필기 정리본 + 강의 슬라이드 원본

※ 손필기에서 오해 소지가 있는 표현은 마지막 "필기 오류 체크" 장에서 슬라이드와 대조해 교정함.

왜 이 순서로 발전했는가?

1. One-Hot	단어를 구분할 수 있지만 의미가 없다.
2. Word Embedding	비슷한 문맥의 단어를 가까운 벡터로 학습한다.
3. RNN	문장의 순서와 이전 정보를 hidden state 로 이어간다.
4. LSTM	게이트와 cell state 로 오래된 정보를 더 잘 보존한다.
5. Seq2Seq	입력 시퀀스를 다른 출력 시퀀스로 변환한다.
6. Attention	하나의 고정 벡터 대신 필요한 encoder 정보를 직접 본다.
7. Self-Attention	RNN 순차 전달 없이 같은 문장 단어끼리 직접 관계를 계산한다.
8. Transformer	Self-Attention 에 위치/비선형/마스킹/잔차 등을 결합한다.
9. BERT / GPT	대규모 텍스트로 언어를 먼저 배우고 여러 태스크에 활용한다.
CORE 핵심 프레임 NLP 모델의 발전은 "더 많은 문맥을 더 직접적으로, 더 병렬적으로 활용하려는 과정"으로 볼 수 있다.	

CHAPTER 1 1. 단어를 숫자로 표현하기

1-1. One-Hot Encoding

DEF 한 줄 정의

각 단어를 "한 위치만 1, 나머지는 0"인 벡터로 표현하는 방식.

예시: 단어 사전 [사과, 바나나, 자동차, 버스]

단어	One-Hot 벡터
사과	[1, 0, 0, 0]
바나나	[0, 1, 0, 0]
자동차	[0, 0, 1, 0]
버스	[0, 0, 0, 1]

- 왜 필요했나? 신경망은 문자열을 직접 계산하지 못하므로 단어를 숫자로 바꿀 필요가 있었다.
- 쉬운 비유: 단어마다 주민등록번호를 하나씩 붙이는 것. "누구인지"는 알지만 "어떤 사람인지"는 모른다.
- 한계 1 - 의미적 정보 부족: 은행/금융처럼 관련 있는 단어도 서로 완전히 독립적인 벡터가 된다.
- 한계 2 - 차원의 저주: 어휘가 50 만 개면 벡터 차원도 50 만 차원이 되어 매우 희소(sparse)하다.

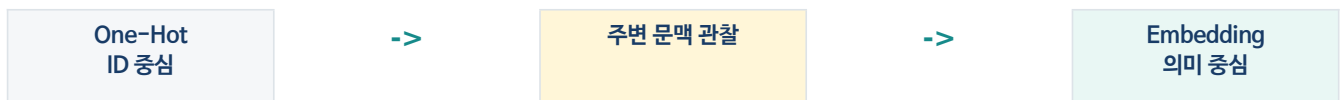
SLIDE 슬라이드 연결

강의 p.1-2: One-Hot 벡터는 서로 직교하기 쉬워 유사도를 반영하기 어렵고, 고차원 희소 표현이라는 문제가 있음을 강조한다.

1-2. Word Embedding

DEF 한 줄 정의

단어 사이의 의미적 관계를 담을 수 있도록, 작은 차원의 밀집된(dense) 실수 벡터로 표현하는 방법.



- 핵심 아이디어: "단어의 의미를 알고 싶으면 그 단어 주변에 어떤 단어가 오는지 보라."
- 비슷한 문맥에 자주 등장하는 단어는 비슷한 벡터 위치를 갖도록 학습될 수 있다.
- 쉬운 비유: 주민등록번호 대신 취향/성격/직업 같은 여러 특징 좌표로 사람을 배치하는 것.

MEMO 기억 문장

One-Hot 은 "단어의 ID"를 표현하고, Word Embedding 은 "단어의 의미적 위치"를 학습한다.

1-3. Word2Vec: Skip-gram vs CBOW

구분	Skip-gram	CBOW
방향	중심 단어 -> 주변 단어	주변 단어 -> 중심 단어
예	bank -> money, loan	money, loan -> bank
속도	상대적으로 느림	상대적으로 빠름
강점	적은 데이터/희귀 단어에 상대적으로 강함	자주 등장하는 단어 학습이 효율적
주의	Encoder 가 아님	Decoder 가 아님

CHAPTER 2 2. 순서를 기억하기: RNN 과 LSTM

2-1. 순차적 데이터는 무엇이 다른가?

특징	의미	예
순서	데이터의 배치 순서가 의미를 바꿈	"나는 너를 사랑해" vs "너는 나를 사랑해"
장기 의존성	멀리 떨어진 과거 정보가 현재 판단에 영향	문장 앞의 주어가 뒤의 동사와 연결
가변 길이	문장마다 토큰 수가 다름	짧은 문장/긴 문장 모두 존재

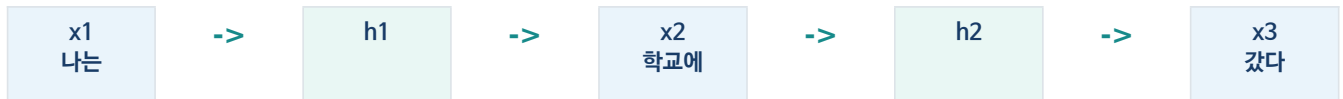
PROBLEM 이전 방법의 한계

일반적인 MLP 는 고정 길이 벡터 처리에 적합하고, 자연스럽게 시간 순서를 전달하는 내부 메모리가 없다.

2-2. RNN (Recurrent Neural Network)

DEF 한 줄 정의

현재 입력과 이전 hidden state 를 함께 사용하여 순차 데이터를 처리하는 신경망.



RNN 핵심식

$$h_t = \tanh(W_{hh} h_{(t-1)} + W_{xh} x_t)$$

현재 입력 x_t 와 이전 기억 $h_{(t-1)}$ 을 합쳐 새로운 기억 h_t 를 만든다. 시간마다 같은 가중치를 공유한다.

- 쉬운 비유: 릴레이 달리기. 앞 시점이 정보를 담은 바통(hidden state)을 다음 시점으로 넘긴다.
- 장점: 가변 길이 시퀀스를 처리할 수 있고, 과거 입력이 현재 출력에 영향을 줄 수 있다.
- 한계: 멀리 있는 정보가 여러 recurrence 단계를 거쳐야 해서 장기 의존성 학습이 어렵다.

2-3. Vanishing Gradient



LIMIT 왜 생기나?

역전파에서 작은 값들이 여러 시점에 걸쳐 반복해서 곱해지면, 앞쪽 시점으로 갈수록 gradient 가 거의 0 이 된다. 결과적으로 먼 과거의 패턴을 학습하기 어렵다.

직관 예: $0.5 \times 0.5 \times 0.5 \times \dots \rightarrow$ 빠르게 0 에 가까워짐

2-4. LSTM (Long Short-Term Memory)

DEF 한 줄 정의

RNN 에 Cell State 와 Gate 를 추가하여 장기 정보를 선택적으로 저장/삭제/출력하는 구조.

구성	역할	비유
Hidden State h_t	당장 다음 계산에 사용할 문맥	지금 머릿속에 떠오른 내용
Cell State C_t	상대적으로 장기간 유지되는 정보 통로	메모장
Forget Gate	과거 정보 중 버릴 양 결정	메모에서 지우기
Input Gate	새 정보 중 기록할 양 결정	새 내용 적기
Output Gate	Cell State 중 밖으로 보여줄 양 결정	지금 읽어줄 부분 선택

Cell State 업데이트

$$C_t = f_t * C_{(t-1)} + i_t * \tilde{C}_t$$

과거 중 남길 정보 + 새로 기록할 후보 정보를 합쳐 새 Cell State 를 만든다.

CORE RNN -> LSTM 의 핵심 변화

RNN 은 "기억을 계속 덮어쓰기"에 가깝다면, LSTM 은 "버릴 것/쓸 것/내보낼 것을 게이트로 선택"한다.

RNN vs LSTM 한눈에 보기

구분	RNN	LSTM
기억 구조	Hidden State	Hidden State + Cell State
정보 선택	단순 업데이트	Forget/Input/Output Gate
장기 의존성	취약	상대적으로 개선
비유	바통 전달	메모장이 있는 바통 전달

CHAPTER 3 3. 문장을 만들고 변환하기: LM 과 Seq2Seq

3-1. Language Model (LM)

DEF 한 줄 정의

앞의 문맥이 주어졌을 때 다음 단어(토큰)의 확률을 예측하고, 문장 전체에 확률을 부여하는 모델.

예: "나는 오늘 학교에 ____"

후보	예측 확률(예시)
갔다	0.72
왔다	0.11
컴퓨터	0.001

문장 확률

$$P(w_1, \dots, w_n) = P(w_1) * P(w_2|w_1) * \dots * P(w_n|w_1, \dots, w_{(n-1)})$$

문장의 확률을 "지금까지의 단어가 주어졌을 때 다음 단어의 조건부 확률"들의 곱으로 표현한다.

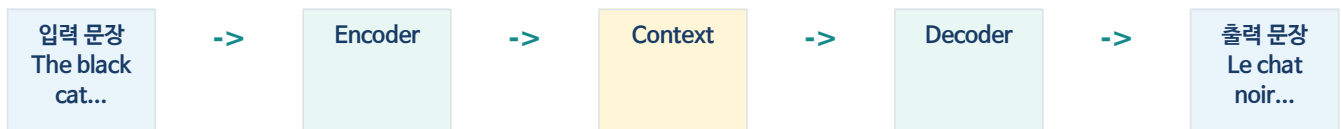
FIX 표기 교정

손필기의 "LMM"은 표준적인 표현이 아니다. 이 부분은 LM(Language Model)로 쓰는 것이 정확하다. LLM은 Large Language Model을 뜻한다.

3-2. Seq2Seq

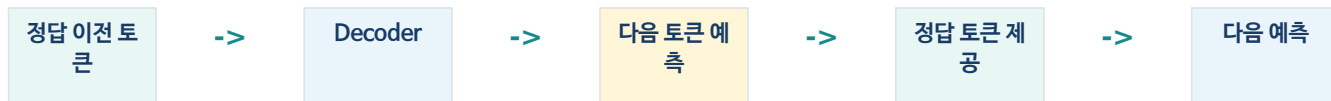
DEF 한 줄 정의

Encoder가 입력 시퀀스를 표현으로 바꾸고, Decoder가 그 표현을 조건으로 출력 시퀀스를 생성하는 구조.



- 왜 필요했나? 번역처럼 입력 길이와 출력 길이가 서로 다른 sequence-to-sequence 문제를 처리하기 위해.
- 응용: 기계 번역, 요약, 대화, 코드 생성 등.
- 초기 Seq2Seq는 Encoder의 마지막 hidden state 같은 고정 길이 벡터에 입력 문장 전체 정보를 압축했다.

3-3. Teacher Forcing



TRAIN 학습에서 하는 일

모델이 직전에 예측한 토큰 대신 실제 정답 토큰을 다음 Decoder 입력으로 넣어 안정적이고 빠르게 학습한다.

FIX 대표 단점은 "과적합"이 아님

훈련에서는 항상 정답 이전 토큰을 보지만, 추론에서는 자기 예측을 다시 입력으로 사용한다. 이 차이를 Exposure Bias 또는 Train-Inference Mismatch 로 이해하는 것이 정확하다.

3-4. 추론: Greedy vs Beam Search

방법	아이디어	장단점
Greedy	매 단계 가장 확률 높은 토큰 1 개 선택	빠르지만 한 번의 잘못된 선택을 되돌리기 어려움
Beam Search	매 단계 상위 k 개 후보 시퀀스를 유지	계산은 늘지만 전체 문장 후보를 더 넓게 탐색

FIX 추론 교정

Autoregressive Decoder 는 일반적으로 "한 번에 문장 전체를 생성"하지 않는다. 한 토큰씩 순차적으로 생성하고, 방금 생성한 토큰이 다음 입력이 된다.

CHAPTER 4 4. 필요한 정보를 직접 찾기: Attention

4-1. Seq2Seq Bottleneck



PROBLEM 문제

문장이 길어져도 모든 의미를 하나의 고정 길이 벡터에 압축하면 정보 손실이 커질 수 있다. 이것이 Seq2Seq의 bottleneck 문제다.

4-2. Attention의 핵심 아이디어

DEF 한 줄 정의

출력 토큰을 만들 때, Encoder의 모든 hidden state 중 지금 필요한 부분에 높은 가중치를 주어 직접 참고하는 메커니즘.

예: Decoder가 프랑스어 "chat"을 만들려고 할 때

입력 단어	Attention 가중치(예시)
The	0.05
black	0.20
cat	0.65
drank	0.05
milk	0.05

Context Vector

$$c_t = \sum_i \alpha(t, i) * h_i$$

각 Encoder hidden state h_i 에 중요도 $\alpha(t, i)$ 를 곱해서 더한다. 중요한 단어의 정보가 더 많이 반영된다.

- Bottleneck 완화: 마지막 hidden state 하나에만 의존하지 않고 모든 encoder state에 접근.
- 긴 문장 성능 개선: 멀리 있는 단어를 직접 참고할 수 있음.
- Alignment/해석 가능성: 출력 토큰을 만들 때 어떤 입력 위치를 강하게 봤는지 관찰할 수 있음.

4-3. Query, Key, Value

개념	역할	검색 비유
Query (Q)	무엇을 찾고 싶은지 표현	검색창에 입력한 검색어
Key (K)	각 정보가 어떤 특징을 가졌는지 표현	문서의 색인/키워드
Value (V)	실제로 가져올 정보 내용	검색 결과 본문



Attention 기본식

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T) V$$

점수는 Query와 Key 사이에서 계산하고, 그 가중치로 Value를 합친다.

CHAPTER 5 5. RNN 을 없애기: Self-Attention 과 Transformer

5-1. 왜 RNN 을 계속 써야 할까?

RNN 의 한계	왜 문제인가
장기 의존성	멀리 떨어진 두 단어가 상호작용하려면 여러 recurrence 단계를 거쳐야 함
병렬화 어려움	h_t 계산에 $h_{(t-1)}$ 이 필요하므로 시점 순서대로 계산해야 함

IDEA 다음 아이디어

Attention 이 필요한 정보를 직접 볼 수 있다면, 아예 문장 내부 관계 계산을 Attention 중심으로 만들 수 있지 않을까?

5-2. Self-Attention

DEF 한 줄 정의

한 문장 안에서 각 단어가 같은 문장의 모든 단어를 직접 참고해 새로운 표현을 만드는 Attention.

예: "어제 카페에 갔는데 거기 사람이 많았다"에서 "거기"는 "카페"와 직접 강하게 연결될 수 있다.

기존 Seq2Seq Attention	Self-Attention
Decoder 가 Encoder 를 봄	문장 내부 단어가 같은 문장 단어를 봄
서로 다른 두 시퀀스 사이 관계	한 시퀀스 내부 관계
출력 시점마다 입력 정보를 검색	각 토큰 표현을 문맥적으로 다시 계산

5-3. 전체 계산량 vs 순차 단계 수

CAUTION 가장 중요한 구분

Self-Attention 은 모든 단어쌍 관계를 계산하므로 전체 attention 계산량은 일반적으로 시퀀스 길이 n 에 대해 $O(n^2)$ 이다. 다만 RNN 처럼 n 개의 시점을 순차적으로 거칠 필요가 없고, 한 층에서 토큰들을 병렬로 계산할 수 있다.

구분	RNN	Self-Attention
순차적으로 거쳐야 할 단계	$O(n)$	$O(1)$ (한 layer 기준)
토큰 간 최대 상호작용 거리	$O(n)$	$O(1)$
대표 계산 특성	순차 의존	모든 토큰쌍 관계로 $O(n^2)$

FIX 필기 문장 교정

"연산 수가 시퀀스 길이에 따라 증가하지 않는다"가 아니라, "순차적으로 거쳐야 할 연산 단계 수가 길이에 따라 증가하지 않는다"가 더 정확하다.

5-4. Scaled Dot-Product Attention

Transformer Attention

$$\text{softmax}(QK^T / \sqrt{d_k}) V$$

QK^T 값이 차원이 커질수록 너무 커져 softmax 가 극단적으로 치우치는 것을 줄이기 위해 $\sqrt{d_k}$ 로 나눠 스케일을 맞춘다.

- Q: 이 토큰이 무엇을 찾을지
- K: 각 토큰이 어떤 특성을 가지고 있는지
- V: 실제로 전달할 정보
- 결과: 각 토큰은 문맥을 반영한 새로운 벡터로 바뀐다.

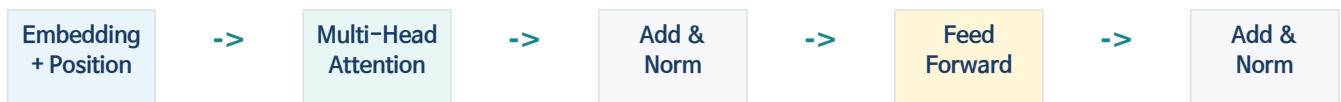
5-5. Self-Attention 만으로 부족한 점

문제	해결책	이유
순서 정보 부재	Positional Encoding	Attention 자체는 토큰 위치 순서를 자동으로 구분하지 않음
비선형성 부족	Feed-Forward Network	가중합 이후 비선형 변환으로 표현력을 확장
미래 참조 문제	Masked Self-Attention	생성 시 미래 토큰을 보지 못하게 score 를 $-\infty$ 처리

5-6. Transformer 전체 구조

DEF 한 줄 정의

RNN recurrence 를 없애고 Self-Attention 을 중심으로, Positional Encoding, FFN, Residual Connection, Layer Normalization 등을 결합한 신경망 구조.



Multi-Head Attention

CORE 핵심 목적

여러 Attention head 가 서로 다른 관계(문법, 의미, 지시 대상 등)를 병렬로 학습하도록 한다.

Head 예시	집중할 수 있는 관계(개념적 예)
Head 1	문법적 의존성
Head 2	대명사-선행사 관계
Head 3	의미적 연관성

FIX 파라미터 오해 교정

512 차원을 8개 head x 64 차원으로 나누면 각 head 의 행렬은 작아지지만 head 가 여러 개 존재한다. Multi-Head 의 핵심은 전체 파라미터를 단순히 줄이는 것이 아니라 다양한 표현 공간에서 관계를 동시에 학습하는 것이다.

Residual Connection + Layer Normalization

구성	역할
Residual Connection	입력 x 를 sublayer 출력과 더해 깊은 네트워크의 학습 경로를 안정화
Layer Normalization	각 토큰의 hidden vector 값을 정규화하여 학습을 안정화

Encoder / Decoder / Cross-Attention

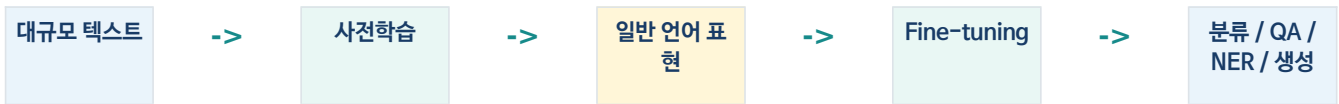
구성	Self-Attention	특징
Transformer Encoder	양방향으로 입력 전체를 볼 수 있음	입력 이해/표현 생성
Transformer Decoder	미래를 마스킹한 Self-Attention	왼쪽 문맥으로 다음 토큰 생성
Cross-Attention	Q는 Decoder, K/V는 Encoder에서 가져옴	Decoder가 Encoder 표현을 참고

MEMO 한 줄 기억

Encoder = 입력을 이해하는 블록 / Decoder = 지금까지의 출력으로 다음 토큰을 만드는 블록 / Cross-Attention = Decoder가 Encoder 결과를 검색하는 연결.

CHAPTER 6 6. 매번 처음부터 학습하지 않기: 사전학습

6-1. Pretraining -> Fine-tuning



PROBLEM 왜 필요했나?

Word Embedding 만 미리 학습하면 첫 층의 단어 표현만 가져오고 나머지 네트워크는 새로 배워야 한다. 언어모델 전체를 사전학습하면 훨씬 풍부한 문맥 지식과 파라미터를 재사용할 수 있다.

- 사전학습: 인터넷 등 대규모 텍스트에서 일반적인 언어 패턴과 표현을 학습.
- Fine-tuning: 사전학습된 파라미터를 특정 다운스트림 태스크에 맞게 조금 더 학습.
- 강의에서는 Encoder / Encoder-Decoder / Decoder 계열의 사전학습 모델을 구분한다.

6-2. BERT

DEF 한 줄 정의

Transformer Encoder 기반으로 양방향 문맥을 활용하며, Masked Language Modeling 으로 사전학습하는 대표적인 모델.

빈칸 맞추기 직관: "나는 오늘 [MASK]에 갔다."에서 앞과 뒤 문맥을 모두 보며 [MASK] 자리를 예측한다.

Masked Language Modeling (MLM)

단계	비율	처리
입력 토큰 중 예측 대상 선택	15%	이 토큰들을 원래 단어로 맞히는 학습 대상에 포함
선택된 토큰 중	80%	[MASK]로 치환
선택된 토큰 중	10%	랜덤 토큰으로 치환
선택된 토큰 중	10%	원래 토큰 그대로 유지

WHY 왜 80/10/10 인가?

모든 예측 대상을 [MASK]로만 바꾸면 실제 사용 환경에는 없는 [MASK] 토큰에 과도하게 익숙해질 수 있다. 랜덤 치환/원문 유지도 섞어 더 강건한 표현을 학습한다.

Next Sentence Prediction (NSP)

강의 슬라이드에서는 두 문장을 입력으로 받아, 두 번째 문장이 첫 번째 문장의 실제 다음 문장인지 예측하는 과제로 설명한다.

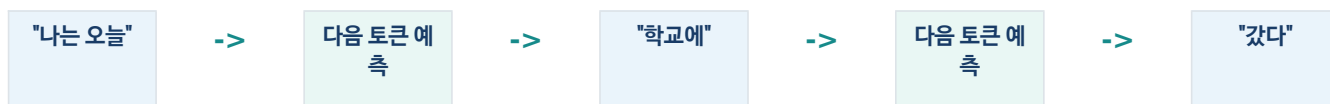
SLIDE 슬라이드 연결

강의 p.44: 입력 토큰의 15%를 선택하고 80/10/10 규칙을 적용. p.45: NSP 와 [CLS]/[SEP] 구조를 소개한다.

6-3. GPT

DEF 한 줄 정의

Transformer Decoder 계열 구조로 이전 토큰들만 보고 다음 토큰을 예측하는 방식으로 사전학습하는 생성형 언어모델.



Autoregressive LM

$$P(x_1, \dots, x_n) = \text{product}_t P(x_t | x_1, \dots, x_{(t-1)})$$

지금까지 나온 토큰들만 조건으로 다음 토큰을 하나씩 예측한다.

SOURCE 자료 범위 메모

손필기에는 Decoder 모델의 "다음 단어 예측"과 In-Context Learning 이 적혀 있다. 제공된 슬라이드 렌더에서는 BERT/NSP 이후 페이지 내용이 명확히 보이지 않으므로, 여기서는 손필기에서 확인되는 범위 중심으로 정리했다.

BERT vs GPT 빠른 비교

구분	BERT	GPT
기본 계열	Transformer Encoder	Transformer Decoder 계열
문맥	양방향	왼쪽 -> 오른쪽
사전학습	Masked LM	Next Token Prediction
직관	빈칸 맞추기	초강력 자동완성
전통적 강점	이해/분류	생성

CHAPTER 7 7. 헛갈리는 개념 비교표

7-1. Skip-gram vs CBOW

구분	Skip-gram	CBOW
입력	중심 단어	주변 단어 묶음
출력	주변 단어	중심 단어
속도	상대적으로 느림	빠름
희귀 단어	상대적으로 강함	상대적으로 약함
핵심 암기	center → context	context → center

7-2. RNN vs LSTM

구분	RNN	LSTM
주요 상태	Hidden State	Hidden State + Cell State
정보 제어	단순 업데이트	Forget/Input/Output Gate
장기 의존성	취약	상대적으로 개선
구조	단순	더 복잡
비유	바통 전달	메모장이 있는 바통 전달

7-3. Seq2Seq vs Attention vs Self-Attention

구분	Seq2Seq	Attention	Self-Attention
정체	모델 구조	메커니즘	메커니즘
관계	Encoder → Decoder	Decoder 가 Encoder 검색	같은 시퀀스 토큰끼리 검색
해결 문제	시퀀스 → 시퀀스 변환	고정 벡터 Bottleneck	RNN 의 장거리/병렬화 한계
비유	통역 시스템	필요한 책 페이지 찾기	단체 채팅방

7-4. BERT vs GPT

구분	BERT	GPT
기본 계열	Transformer Encoder	Transformer Decoder 계열
문맥	양방향	왼쪽 → 오른쪽
대표 사전학습	Masked LM	Next Token Prediction
직관	빈칸 맞추기	초강력 자동완성
전통적 강점	이해/분류/NER/QA	생성/대화/코드
미래 참조	입력 전체 사용 가능	Causal Mask 로 미래 차단

CHAPTER 8. 손필기 오류/오해 소지 체크

아래는 손필기 표현을 강의 슬라이드와 대조하여, 시험이나 복습에서 오해하기 쉬운 지점을 교정한 표다.

필기 내용	정확한 설명	왜 다른가
"LMM"	LM(Language Model)로 표기. LLM 은 Large Language Model.	슬라이드 p.15-17 은 "언어모델"로 설명하며 LMM 은 표준 약어가 아님.
Skip-gram=인코더 / CBOW=디코더	둘은 Word2Vec 의 서로 다른 예측 방식. SG: 중심->주변, CBOW: 주변->중심.	슬라이드 p.4-6 도 두 알고리즘을 비교할 뿐 Encoder/Decoder 한 쌍으로 정의하지 않음.
Teacher Forcing 단점=과적합	대표 문제는 Exposure Bias / Train-Inference Mismatch.	훈련은 정답 이전 토큰, 추론은 모델 자신의 이전 예측을 입력으로 받기 때문.
Self-Attention 은 길어져도 연산 수 증가 X	순차 단계 수는 $O(1)$ 이지만 전체 attention pair 계산은 일반적으로 $O(n^2)$.	슬라이드 p.28 의 "순차적으로 처리해야 하는 연산 수"라는 표현과 전체 계산량을 구분해야 함.
Multi-head 는 matrix/parameter 감소	핵심은 여러 표현 공간에서 다양한 관계를 동시에 학습하는 것.	head 당 차원은 작아져도 head 가 여러 개이므로 전체 파라미터가 단순히 감소한다고 보면 안 됨.
BERT: 15%를 Mask	15%를 예측 대상으로 선택한 뒤 그중 80% [MASK], 10% 랜덤, 10% 그대로.	슬라이드 p.44 에 80/10/10 규칙이 명시되어 있음.

8-1. 추가로 고치면 좋은 3 개

필기 표현	교정
Attention: Query 와 Values 사이 유사도	score 는 Q 와 K 사이에서 계산하고, 그 가중치로 V 를 합친다.
Seq2Seq inference 는 한번에 생성	Autoregressive decoder 는 보통 한 토큰씩 순차 생성한다.
LSTM: $C_t = f_t * C_{(t-1)} + i_t * C_t$	두 번째 C_t 는 후보 Cell State $\tilde{C}_t$ 로 구분해야 한다.

TIP 정확성 우선 암기

시험에서 "왜"를 설명할 때는 키워드 하나보다 원인-해결 관계를 말하는 것이 안전하다. 예: "Teacher Forcing 은 과적합"보다 "학습/추론 입력 분포가 달라 Exposure Bias 가 생긴다".

CHAPTER 9 9. 초보자가 자주 헷갈리는 Q&A

Q&A Q1. Word Embedding 과 Hidden State 는 둘 다 벡터인데 뭐가 다른가?

Embedding 은 단어의 기본 표현이고, Hidden State 는 앞뒤/과거 문맥을 반영해 시점별로 달라지는 문맥 표현이다.

Q&A Q2. LSTM 이면 Attention 이 필요 없는가?

아니다. LSTM 은 장기 기억을 개선하지만 정보가 여전히 시점 간 전달된다. Attention 은 필요한 위치에 직접 접근하도록 만든다.

Q&A Q3. Attention 과 Self-Attention 의 차이는?

Seq2Seq Attention 은 보통 Decoder 가 Encoder 정보를 검색하고, Self-Attention 은 같은 시퀀스 안의 토큰끼리 서로를 검색한다.

Q&A Q4. Transformer 는 항상 RNN 보다 계산량이 적은가?

그렇다고 단정할 수 없다. Self-Attention 은 길이 n 에 대해 $O(n^2)$ 관계 계산을 하지만, 순차 의존이 적어 GPU 병렬화가 훨씬 쉽다는 점이 핵심이다.

Q&A Q5. BERT 와 GPT 는 둘 다 Transformer 인데 왜 다르게 동작하나?

사용하는 구조와 학습 목표가 다르기 때문이다. BERT 는 Encoder/양방향/MLM, GPT 는 Decoder 계열/causal mask/다음 토큰 예측이 핵심이다.

9-1. Q/K/V 를 한 번 더

Q	K	V
무엇을 찾을까?	내가 어떤 정보인가?	실제로 전달할 내용

검색 비유: 검색어(Query)로 색인(Key)을 비교해 관련도를 계산하고, 관련도가 높은 문서 내용(Value)을 가져온다.

FINAL 10. 시험 직전 1 페이지 압축 정리

1. One-Hot: 단어를 구분하지만 의미 관계와 효율성이 부족하다.
2. Word Embedding: 주변 문맥으로 단어 의미를 밀집 벡터에 담는다.
3. RNN: hidden state 로 과거 정보를 순차적으로 전달한다.
4. LSTM: cell state + gate 로 장기 의존성 학습을 개선한다.
5. Language Model: 이전 문맥으로 다음 토큰 확률을 예측한다.
6. Seq2Seq: Encoder 가 입력을 표현하고 Decoder 가 출력 시퀀스를 생성한다.
7. Attention: Decoder 가 필요한 Encoder 위치를 직접 참고해 bottleneck 을 완화한다.
8. Self-Attention: 같은 문장 내 토큰들이 서로 직접 관계를 계산하며 병렬화가 쉬워진다.
9. Transformer: Self-Attention + Position + FFN + Masking + Residual/Norm 을 결합한다.
10. BERT: Encoder + 양방향 + MLM(15% 선택, 80/10/10).
11. GPT: Decoder 계열 + causal mask + 다음 토큰 예측으로 생성에 강하다.

ONE LINE 최종 암기 문장

Embedding → RNN → LSTM → Seq2Seq → Attention → Self-Attention → Transformer → BERT/GPT 는 "더 긴 문맥을 더 직접적이고 더 병렬적으로 활용하는 방향"으로 발전한 흐름이다.

자료 기준 및 주의

- 손필기 자료: ai decoder.pdf (7p) - 학습자의 정리 내용을 기반으로 오류/오해 소지를 체크함.
- 강의 슬라이드: ai LLM.pdf (52p) - One-Hot, Word2Vec, RNN/LSTM, Seq2Seq/Attention, Self-Attention/Transformer, BERT 슬라이드 내용을 기준으로 구성함.
- GPT 부분은 손필기에서 확인되는 "Decoder 모델 / 다음 단어 예측 / In-Context Learning" 범위를 중심으로 정리했으며, 제공된 슬라이드 렌더에서 해당 뒷부분이 명확히 확인되지 않는 점을 문서에 표시함.